FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

Module : Micro Services

Pr.CHEMLAL Yman

Cahier des Charges : Développement d'un Microservice Industriel

Objectif du Projet

L'objectif est de concevoir et réaliser un microservice autonome et robuste permettant la gestion complète d'une **[Ressource]**. Ce projet doit démontrer votre maîtrise de l'écosystème **Spring Boot** et votre capacité à exposer une API conforme aux standards **RESTful** utilisés en entreprise.

1. Exigences Fonctionnelles (CRUD & Logique)

Le microservice doit permettre d'effectuer les opérations suivantes :

- **Création** : Ajouter une nouvelle instance de [Ressource] via une requête POST.
- **Lecture** : Récupérer la liste complète des instances ou une instance spécifique par son ID unique.
- **Mise à jour** : Modifier les informations d'une [Ressource] existante (PUT).
- **Suppression** : Supprimer une [Ressource] de la base de données (DELETE).
- **Recherche Métier** : Implémenter au moins une méthode de recherche personnalisée (ex: filtrage par prix, par nom, ou par statut).

NB :

- **Définition : Dans une API REST, une ressource est n'importe quel objet ou information que l'on peut nommer, manipuler et identifier de manière unique.**
- **Analogie :**
  - **Sur un ordinateur, la ressource est le Fichier.**
  - **Dans une base de données, la ressource est l'Enregistrement (la ligne).**
  - **Dans votre microservice, la ressource est votre Objet Métier.**

2. Contraintes Techniques (Architecture & Qualité)

Votre projet devra impérativement intégrer les éléments suivants :

- **Persistance de données** : Utilisation de **Spring Data JPA** avec une base de données **H2** (en mémoire).

- **Initialisation SQL** : Fournir les fichiers schema.sql (création des tables) et data.sql (insertion de données de test) pour un démarrage automatique.
- **Conformité REST** :
  - Utilisation des codes de statut HTTP appropriés (ex: **201 Created** pour un ajout réussi, **404 Not Found** si la ressource n'existe pas).
  - Utilisation de **ResponseEntity** pour contrôler les en-têtes et le corps des réponses.
- **Gestion des Exceptions** : Création d'une classe d'exception personnalisée et d'un gestionnaire global (**@ControllerAdvice**) pour renvoyer des messages d'erreurs clairs au format JSON.
- **Validation & Sécurité** :
  - Mise en place de la **Validation des Beans** (@Valid, @Min, @Size...) pour garantir la qualité des données entrantes.
  - Mise en place d'un **Filtrage JSON** dynamique ou statique pour masquer les données sensibles (ex: prix d'achat, mots de passe).
- **Documentation** : L'API doit être entièrement documentée et testable via une interface **Swagger UI** (OpenAPI).

3. Livrables Attendus

Pour valider ce module, l'étudiant doit remettre un dossier compressé contenant :

1. **Le code source Java** structuré proprement en packages (model, dao, controller, exceptions).
2. **Le fichier pom.xml** listant toutes les dépendances nécessaires.
3. **Une collection Postman (JSON)** contenant l'ensemble des requêtes testées (Cas nominaux et cas d'erreurs).
4. **Une courte notice** indiquant l'URL d'accès à Swagger et à la console H2.

Critères d'Évaluation:

- **Architecture & Propreté du code** (Injection par constructeur, respect du modèle MVC) : **5 pts**
- **Maîtrise de JPA & SQL** (Configuration correcte, requêtes personnalisées) : **5 pts**
- **Robustesse** (Validation, gestion des erreurs, codes HTTP) : **5 pts**
- **Documentation & Tests** (Swagger fonctionnel, Collection Postman complète) : **5 pts**

Structure commune à tous les projets

Pour chaque projet, l'étudiant doit fournir :

1. **Le code source** (Architecture propre en packages : model, dao, web.controller, web.exceptions).
2. **Les fichiers SQL** (schema.sql et data.sql).
3. **Une collection Postman** incluant les tests CRUD et les cas d'erreurs.
4. **L'interface Swagger** documentée avec des descriptions claires.

Liste des 20 projets : Chaque projet est structuré de manière à évaluer l'ensemble des compétences acquises : **Spring Boot, REST, JPA (H2), Postman, Validation, Exceptions, Filtrage JSON et Swagger.**

1. Gestion de Stock High-Tech

- **Entité :** Produit (id, nom, marque, prixAchat, prixVente, quantite).
- **Filtrage :** Masquer prixAchat.
- **Requête JPA :** Trouver les produits dont le stock est inférieur à 5.
- **Validation :** PrixVente doit être > PrixAchat.

2. Système de Gestion Universitaire

- **Entité :** Etudiant (id, nom, email, moyenneGenerale, codeSecretPaiement).
- **Filtrage :** Masquer codeSecretPaiement.
- **Requête JPA :** Lister les étudiants ayant une moyenne > 10.
- **Validation :** Email doit être un format valide.

3. Catalogue de Micro-Services Cloud

- **Entité :** Service (id, nomService, url, statut, coutMensuel).
- **Filtrage :** Masquer url technique.
- **Requête JPA :** Trouver les services "Down" (statut).
- **Validation :** Le nom du service doit faire au moins 3 caractères.

4. Gestion d'une Clinique Médicale

- **Entité :** Patient (id, nom, prenom, numeroSecu, diagnosticConfidentiel).
- **Filtrage :** Masquer diagnosticConfidentiel.
- **Requête JPA :** Rechercher un patient par son numéro de sécu.
- **Validation :** NumeroSecu doit contenir exactement 15 chiffres.

5. Agence de Location de Voitures

- **Entité :** Vehicule (id, modele, immatriculation, prixJour, prixAchat).
- **Filtrage :** Masquer prixAchat.
- **Requête JPA :** Trouver les véhicules dont le prix de location est entre 50 et 100€.
- **Validation :** Immatriculation doit suivre le format AA-111-AA.

6. Plateforme de Recrutement (JobBoard)

- **Entité :** OffreEmploi (id, titre, description, salairePropose, budgetMax).
- **Filtrage :** Masquer budgetMax.
- **Requête JPA :** Rechercher les offres par mot-clé dans le titre.
- **Validation :** SalairePropose ne peut pas être nul.

7. Bibliothèque Numérique

- **Entité :** Livre (id, titre, auteur, isbn, prixVente, margeEditeur).
- **Filtrage :** Masquer margeEditeur.
- **Requête JPA :** Trouver tous les livres d'un auteur donné.
- **Validation :** ISBN est obligatoire et unique.

8. Suivi de Portefeuille Crypto

- **Entité :** Crypto (id, nom, symbole, quantiteDetenue, clePrivee).
- **Filtrage :** Masquer clePrivee.
- **Requête JPA :** Trier les cryptos par quantité décroissante.
- **Validation :** Symbole doit être en majuscules (ex: BTC).

9. Réservation de Salles de Coworking

- **Entité :** Salle (id, nom, capacite, prixHeure, coutEntretien).
- **Filtrage :** Masquer coutEntretien.
- **Requête JPA :** Lister les salles pouvant accueillir plus de 20 personnes.
- **Validation :** Capacite doit être entre 1 et 100.

10. Gestion d'un Restaurant (Carte)

- **Entité :** Plat (id, nom, calories, prix, coutIngredients).
- **Filtrage :** Masquer coutIngredients.
- **Requête JPA :** Trouver les plats "Healthy" (< 500 calories).
- **Validation :** Le prix doit être positif.

11. E-Pharmacie (Médicaments)

- **Entité :** Medicament (id, nom, stock, ordonnanceRequise, moleculeActive).

- **Filtrage :** Masquer moleculeActive.
- **Requête JPA :** Lister les médicaments nécessitant une ordonnance.
- **Validation :** Stock ne peut pas être négatif.

12. Immobilier (Vente d'appartements)

- **Entité :** Appartement (id, adresse, surface, prixVente, commissionAgence).
- **Filtrage :** Masquer commissionAgence.
- **Requête JPA :** Trouver les appartements avec surface > 100m^2.
- **Validation :** Adresse obligatoire.

13. Gestion de Serveurs IT

- **Entité :** Serveur (id, adresseIP, ramGo, prixLocation, motDePasseAdmin).
- **Filtrage :** Masquer motDePasseAdmin.
- **Requête JPA :** Trouver les serveurs ayant plus de 64Go de RAM.
- **Validation :** IP doit être au format valide (ex: 192.168.1.1).

14. Application de Fitness (Exercices)

- **Entité :** Exercice (id, nom, muscleCible, difficulte, secretCoach).
- **Filtrage :** Masquer secretCoach.
- **Requête JPA :** Trouver les exercices par muscle cible.
- **Validation :** Difficulte comprise entre 1 et 5.

15. Site de Streaming (Musique)

- **Entité :** Chanson (id, titre, artiste, dureeSecondes, lienFichierSource).
- **Filtrage :** Masquer lienFichierSource.
- **Requête JPA :** Trouver les chansons d'un artiste triées par titre.
- **Validation :** DureeSecondes doit être > 10.

16. Boutique de Fleurs en ligne

- **Entité :** Fleur (id, nom, couleur, prix, saisonnalite).
- **Filtrage :** Aucun (ou masquer le fournisseur).
- **Requête JPA :** Rechercher les fleurs par couleur.
- **Validation :** Le nom de la fleur ne peut pas contenir de chiffres.

17. Gestion de Projets Agiles

- **Entité :** Ticket (id, titre, priorite, tempsEstime, tempsPasse).
- **Filtrage :** Masquer tempsPasse.

- **Requête JPA :** Lister les tickets de priorité "Haute".
- **Validation :** TempsEstime doit être en heures entières.

18. Plateforme d'Hébergement de Vidéos

- **Entité :** Video (id, titre, vues, tailleMo, idUtilisateurCreateur).
- **Filtrage :** Masquer idUtilisateurCreateur.
- **Requête JPA :** Trouver les vidéos les plus populaires (> 1 million de vues).
- **Validation :** Titre ne doit pas dépasser 100 caractères.

19. Gestionnaire de mots de passe (Vault)

- **Entité :** Compte (id, siteWeb, identifiant, motDePasseChiffre, notePrivee).
- **Filtrage :** Masquer motDePasseChiffre.
- **Requête JPA :** Rechercher tous les comptes d'un utilisateur par siteWeb.
- **Validation :** URL du site web doit commencer par https.

20. Smart Home (Objets connectés)

- **Entité :** Appareil (id, nom, piece, consommationWatt, adresseMAC).
- **Filtrage :** Masquer adresseMAC.
- **Requête JPA :** Trouver tous les appareils d'une pièce (Salon, Cuisine...).
- **Validation :** ConsommationWatt ne peut pas être nulle.